Patient Management Application with Secure AWS Storage Architecture

Overview

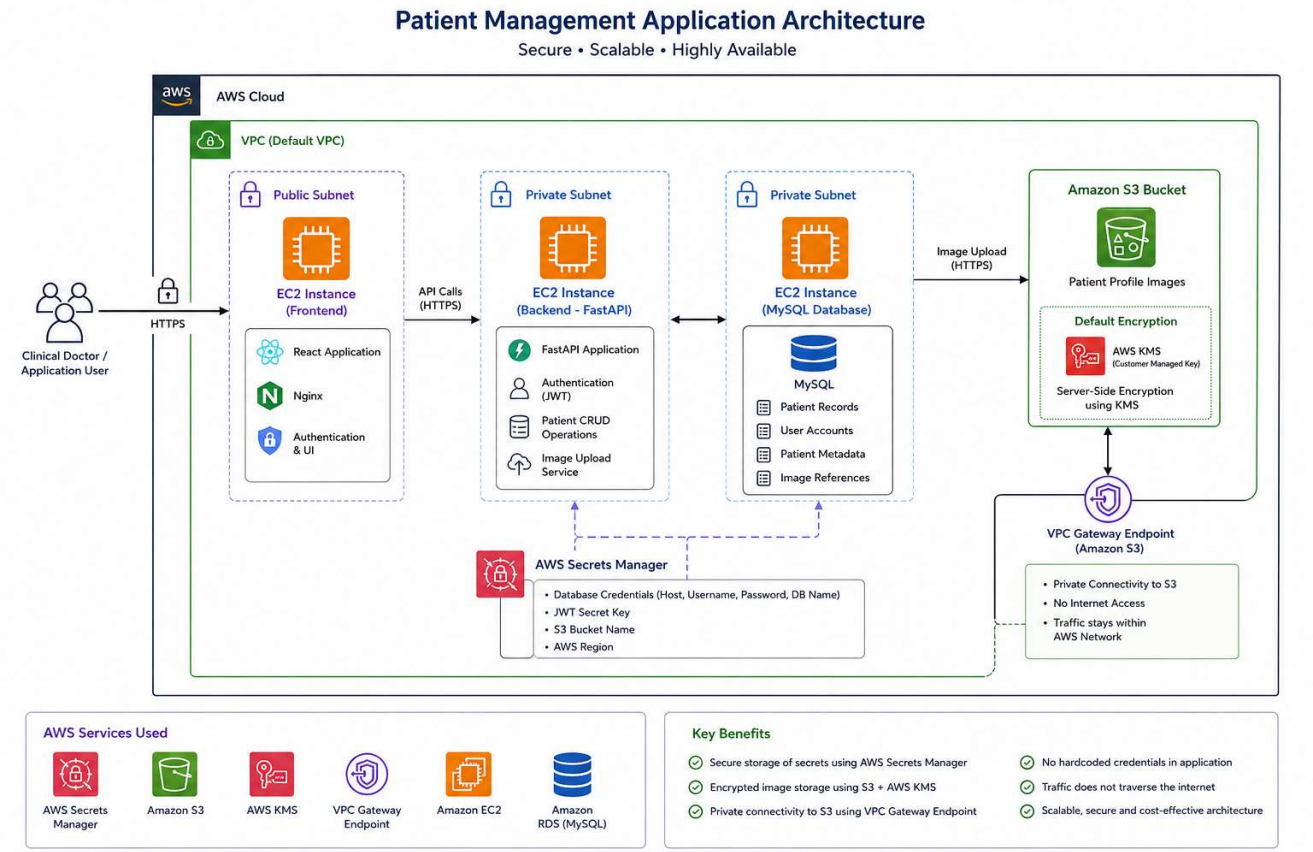
A Patient Management Application was developed to help clinical staff manage and track patient information efficiently. The application provides authentication, patient record management, and secure profile image storage.

The application follows a three-tier architecture where the frontend, backend, and database are deployed on separate Amazon EC2 instances. Security was a primary consideration during the implementation, particularly for handling sensitive configuration data and patient profile images.

The following AWS services were used as part of the solution:

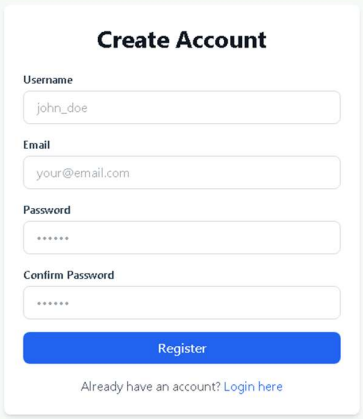
- AWS Secrets Manager
- Amazon S3
- AWS Key Management Service (KMS)
- Amazon VPC Gateway Endpoint for S3
- Amazon EC2

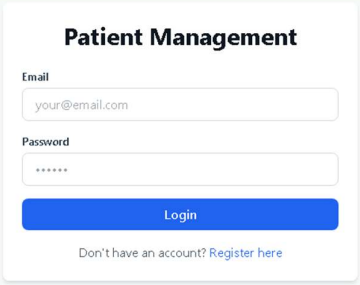
Application Architecture

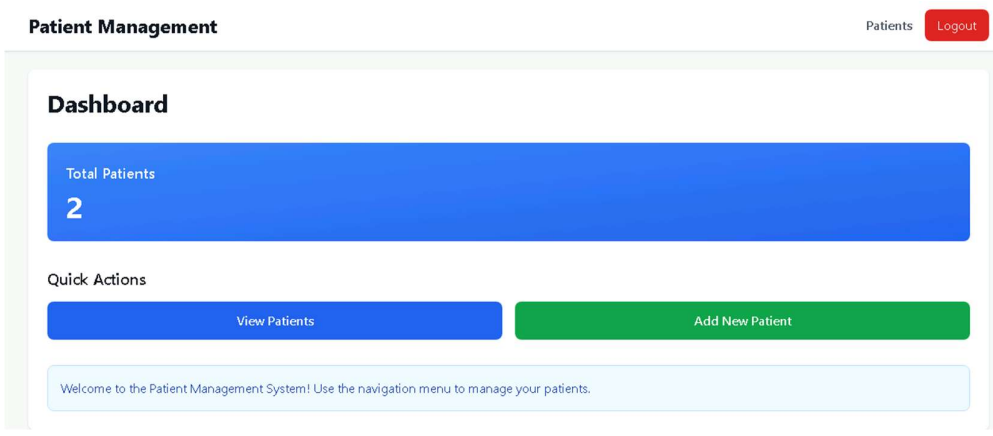


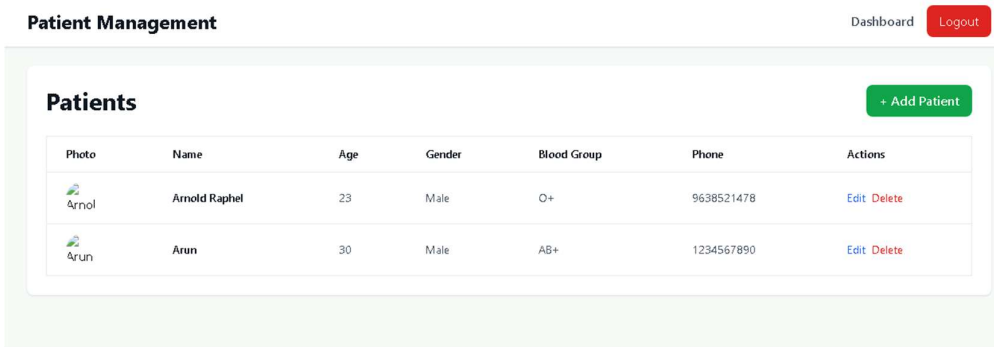
The application consists of:

1. Frontend EC2 Instance
    - Hosts the React frontend.
    - Provides the user interface for authentication and patient management.
2. Backend EC2 Instance
    - Hosts the FastAPI application.
    - Handles authentication, business logic, and S3 integration.
3. Database EC2 Instance
    - Hosts the MySQL database.
    - Stores patient records and application data.
4. Amazon S3
    - Stores patient profile images.
5. AWS Secrets Manager
    - Stores application secrets and credentials.


Create Account
Username
john_doe
Email
your@email.com
Password
Confirm Password
Register
Already have an account? Login here


Patient Management
Email
your@email.com
Password
Login
Don't have an account? Register here


Patient Management
Patients
Logout
Dashboard
Total Patients
2
Quick Actions
View Patients
Add New Patient
Welcome to the Patient Management System! Use the navigation menu to manage your patients.


Patient Management
Dashboard
Logout
Patients
+ Add Patient
Photo
Name
Age
Gender
Blood Group
Phone
Actions
Arnol
Arnold Raphel
23
Male
O+
9638521478
Edit Delete
Arun
Arun
30
Male
AB+
1234567890
Edit Delete

AWS Secrets Manager Integration

Objective

One of the primary security requirements was to avoid storing sensitive information directly in application code or configuration files.

To achieve this, AWS Secrets Manager was used as the centralized secret management solution.

Secrets Stored

The following values are retrieved dynamically from Secrets Manager during application startup:

- Database Host
- Database Username
- Database Password
- Database Name
- JWT Secret Key
- S3 Bucket Name
- AWS Region

This approach eliminates hardcoded credentials and significantly improves security and maintainability.

Benefits

Centralized Secret Management

All application secrets are managed from a single secure location rather than being scattered across configuration files.

Reduced Credential Exposure

Sensitive information is never committed to source control repositories.

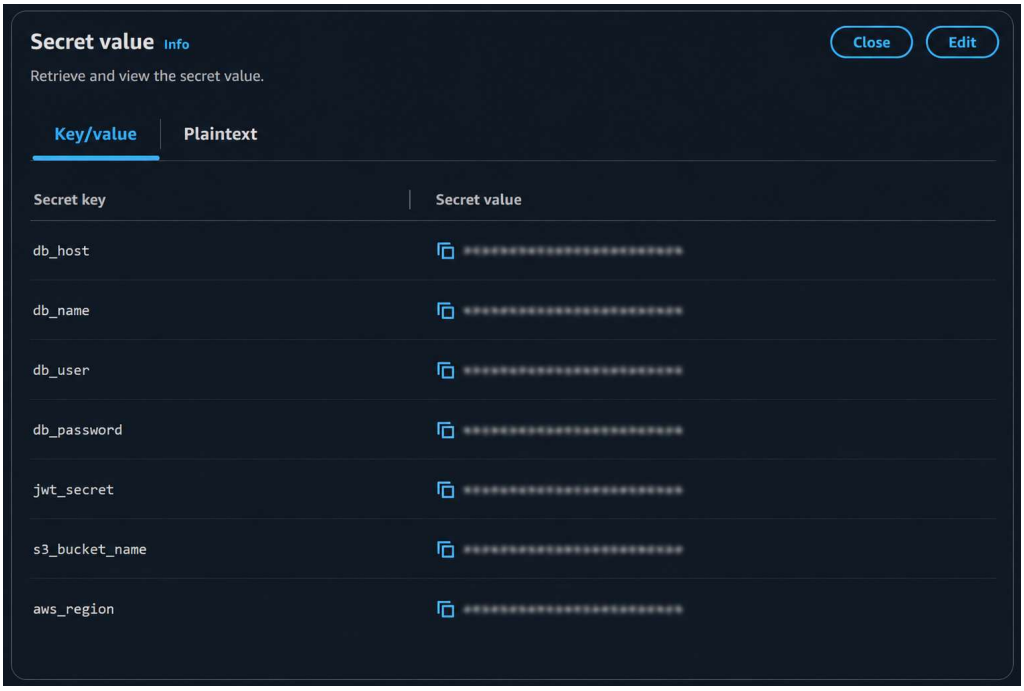
Simplified Credential Rotation

Secrets can be rotated centrally without requiring code modifications.

Enhanced Security

Secrets Manager encrypts stored secrets and provides fine-grained access control through IAM.

Secrets Manager Configuration



Application Secret Retrieval Flow

```python
def get_secret(secret_name: str) -> dict:
    """
    Retrieve a secret from AWS Secrets Manager.
    Returns the secret as a dictionary.
    """
    client = boto3.client('secretsmanager', region_name=settings.aws_region)

    try:
        response = client.get_secret_value(SecretId=secret_name)

        if 'SecretString' in response:
            secret = response['SecretString']
            return json.loads(secret)
        else:
            # Handle binary secrets if needed
            return response['SecretBinary']
    except Exception as e:
        raise RuntimeError(f"Failed to retrieve secret {secret_name}: {str(e)}")
```

```python
def load_secrets_from_manager(secret_name: str):
    """
    Load secrets from AWS Secrets Manager and update settings.
    This should be called during application startup.
    """
    try:
        secrets = get_secret(secret_name)

        # Update settings with secrets
        settings.database_url = (
            f"mysql+pymysql://{secrets.get('db_user')}:{secrets.get('db_password')}"
            f"@{secrets.get('db_host')}/{secrets.get('db_name')}"
        )
        settings.jwt_secret_key = secrets.get('jwt_secret')
        settings.s3_bucket_name = secrets.get('s3_bucket_name')
        if not settings.aws_region:
            settings.aws_region = secrets.get('aws_region', 'us-east-1')

    except Exception as e:
        raise RuntimeError(f"Failed to load secrets: {str(e)}")
```

Secure Image Storage Using Amazon S3 and AWS KMS

Objective

Patient profile images contain potentially sensitive information and therefore require secure storage.

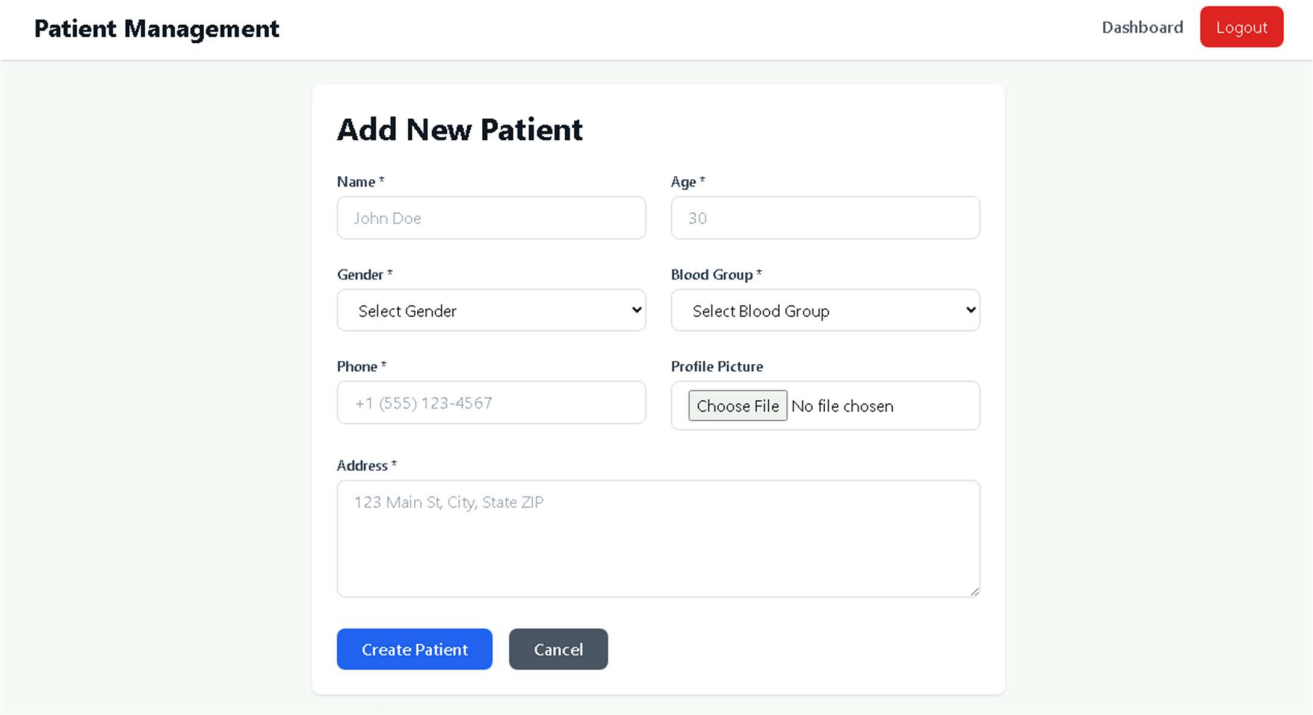
Amazon S3 was selected as the storage service because it provides high durability, scalability, and native integration with AWS security services.

Image Upload Workflow

When a user creates a patient record and uploads a profile image:

1. The frontend sends the image to the backend.
2. The backend uploads the image to Amazon S3.
3. The image is stored in a dedicated S3 bucket.
4. The image metadata or URL is associated with the patient record.

Image Upload Flow



Server-Side Encryption Using AWS KMS

To secure stored images, default bucket encryption was configured using AWS KMS.

Every uploaded image is automatically encrypted before being written to storage.

Encryption and decryption operations are handled transparently by Amazon S3 using the configured KMS key.

Security Benefits

Data Protection at Rest

All profile images remain encrypted while stored in Amazon S3.

Centralized Key Management

Encryption keys are managed through AWS KMS rather than application code.

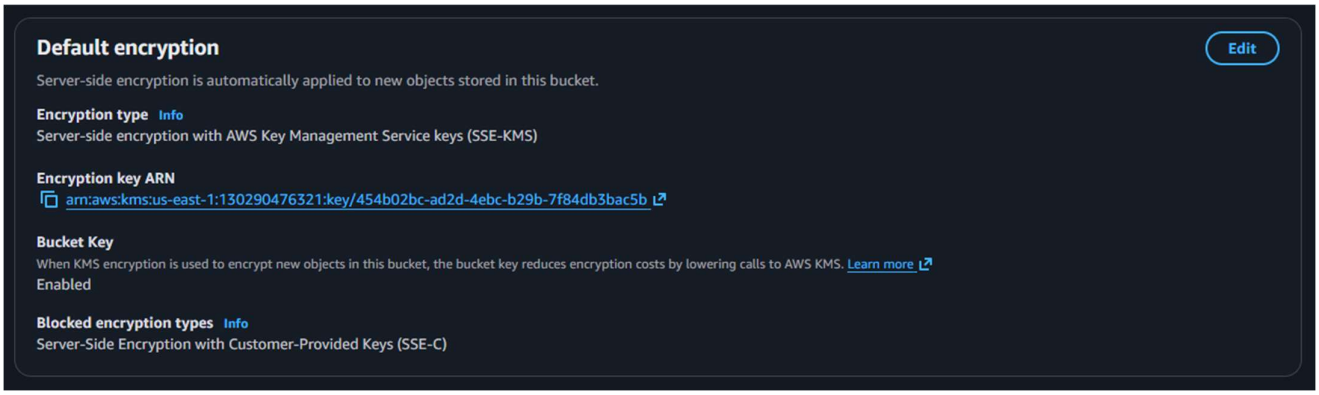
Access Control

KMS permissions can be controlled through IAM policies.

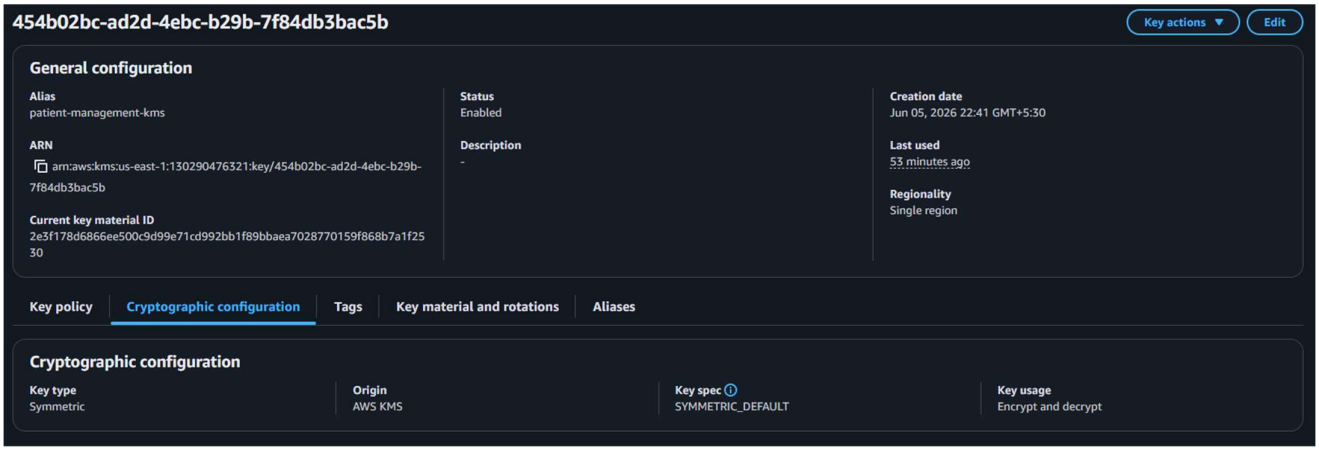
Auditability

KMS usage can be monitored and audited through AWS logging and monitoring services.

S3 Bucket Encryption Configuration



KMS Key Configuration



Private S3 Access Using VPC Gateway Endpoint

Objective

By default, communication between EC2 instances and Amazon S3 may traverse public AWS endpoints.

To improve security, a VPC Gateway Endpoint for Amazon S3 was implemented.

This allows traffic between the backend EC2 instance and Amazon S3 to remain entirely within the AWS network.

Traditional Approach

Without a VPC Endpoint:

EC2 Instance

↓

Internet Gateway

↓

Public S3 Endpoint

↓

Amazon S3

Although traffic remains within AWS infrastructure, it still relies on public endpoint access.

Implemented Approach

With a VPC Gateway Endpoint:

Backend EC2

↓

Private Route Table

↓

S3 Gateway Endpoint

↓

Amazon S3

This removes the requirement for internet-based access when communicating with S3.

Benefits of the Gateway Endpoint

Private Connectivity

Traffic remains within the AWS network and does not require internet access.

Improved Security Posture

Reduces exposure to public network paths.

Better Network Control

Access can be restricted using route tables and bucket policies.

Cost Optimization

Gateway Endpoints do not incur hourly charges unlike Interface Endpoints.

Security Summary

The application was designed with security as a core principle.

The following security controls were implemented:

Security Control	Implementation
Secret Management	AWS Secrets Manager
Authentication	JWT-Based Authentication
Image Storage	Amazon S3
Encryption at Rest	AWS KMS
Private S3 Connectivity	VPC Gateway Endpoint
Infrastructure Isolation	Separate EC2 Instances
Credential Protection	No Hardcoded Secrets

Conclusion

The Patient Management Application demonstrates the integration of multiple AWS security services to protect sensitive healthcare-related data.

AWS Secrets Manager was used to securely manage application credentials and secrets. Amazon S3 was used for scalable profile image storage, while AWS KMS provided encryption for all stored images. Finally, a VPC Gateway Endpoint was implemented to ensure private communication between the backend application and Amazon S3 without requiring internet access.